

P2199R0

Concepts to Differentiate Types

Published Proposal, 2020-07-11

This version:

<https://wg21.link/P2199R0>

Latest published version:

<https://wg21.link/P2199>

Author:

[Isabella Muerte](#)

Audience:

LEWG

Toggle Diffs:

☐ Hide deleted text

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Latest source:

[src/different-types.bs](#)

This source:

[src/different-types.bs](#)

Abstract

The differentiation of types is a commonly used constraint in C++. Typically, wording is copy pasted regarding this behavior, or shown only in exposition only helpers. Adding explicit definitions would reduce the need for multiple users to implement the same types.

Table of Contents

1 Revision History

1.1 Revision 0

2 Motivation

3 Request for Feedback

4 Wording

§ 1. Revision History

§ 1.1. Revision 0

Initial Release. 🤖

§ 2. Motivation

Within the standard library, there are multiple instances of phrases such as "If T is the same type as U , the program is ill-formed", or *This function does not participate in overload resolution unless the following constraints are true*, followed by something like `is_same_v<remove_cvref_t<U>, type>`. It would, honestly, be easier to express these with concepts. Furthermore, the inversion of the `std::same_as` concept is much more common than `std::same_as` itself when trying to constrain signatures of template types. Additionally, in the realm of ranges, users tend to need to express that *similar* types are ok in some instances, while *distinct* (or *different*, see [§ 3 Request for Feedback](#) regarding this phrase) are fairly common. In the author's work codebase, the inversion of `std::same_as` appears about 20 times for each time where `std::same_as` is used. While it is still early days regarding concepts, it would not be surprising if other code bases had similar ratios, though most likely they are not as extreme.

In the interest of symmetry with `same_as` and `different_from`, this paper provides both `similar_to` and `distinct_from`, although the wording found below only redefines wording in terms of `different_from`, and `distinct_from`.

§ 3. Request for Feedback

Currently, there is some ambiguity with the naming of `different_from` and `distinct_from`. When something is *distinct*, it typically implies that it is set apart and completely dissimilar to something else. However, the direct antonym of *same* in english is *different*. The author is fine with

swapping the definition of `different_from` and `distinct_from`, but will defer to any consensus from LEWG or LWG if it is deemed necessary to swap their definitions.

§ 4. Wording

The following wording is based off of the text found at <https://eel.is/c++draft> as of 2020-07-11.

The following is to be modified (according to a value as decided by LWG) in 17.3.2 *Header* `<version>` synopsis `[version.syn]`, statement 2

```
#define __cpp_lib_concepts 202002L 202002L
```

The following is to be added to 18.3 *Header* `<concepts>` synopsis `[concepts.syn]`

```
// [concept.different], concept different from
template <class T, class U>
    concept different_from = see below;

// [concept.similar], concept similar to
template <class T, class U>
    concept similar_to = see below;

// [concept.distinct], concept distinct from
template <class T, class U>
    concept distinct_from = see below;
```

The following is to be added to 18.4 *Language-related concepts* `[concepts.lang]`

```
18.4.❖ Concept                different_from                [concept.different]
template <class T, class U>
    concept different_from = not same_as<T, U>;

18.4.❖ Concept                similar                        [concept.similar]
template <class T, class U>
    concept similar_to = same_as<remove_cvref_t<T>, remove_cvref_t<U>>;

18.4.❖ Concept                distinct_from                  [concept.distinct]
template <class T, class U>
    concept distinct_from = not similar_to<T, U>;
```

The following is to be modified in 20.6.3.1 *Constructors* [**optional.ctor**], statement 22

Constraints: `is_constructible_v<T, U>` is true, ~~`is_same_v<remove_cvref_t<U>, in_place_t>`~~ `distinct_from<U, in_place_t>` is ~~false~~ true, and ~~`is_same_v<remove_cvref_t<U>, optional>`~~ `distinct_from<U, optional>` is ~~false~~ true.

The following is to be modified in 20.6.3.3 *Assignment* [**optional.assign**], statement 14

Constraints: ~~`is_same_v<remove_cvref_t<U>, optional>`~~ `distinct_from<U, optional>` is ~~false~~ true

The following is to be modified in 20.7.3.3 *Assignment* [**variant.assign**], statement 12.1

~~`is_same_v<remove_cvref_t<T>, variant>`~~ `distinct_from<T, variant>` is ~~false~~ true.

The following is to be modified in 20.8.3.1 *Construction and destruction* [**any.cons**], statement 6

Constraints: ~~`VT` is not the same type as `any`~~ `distinct_from<T, any>` is true

The following is to be modified in 20.14.5.1 *Constructors and destructor* [**refwrap.const**], statement 2

Constraints: The expression `FUN(declval<U>())` is well-formed and ~~`is_same_v<remove_cvref_t<U>, reference_wrapper>`~~ `distinct_from<U, reference_wrapper>` is ~~false~~ true

The following is to be modified from 21.2 *Character traits* [**char.traits**], statement 3

To specialize those templates to generate a string, string view, or iostream class to handle a particular character container type ([defns.character.container]) `C`, that and its related character traits class `X` are passed as a pair of parameters to the string, string view, or iostream template as parameters `charT` and `traits`. ~~If `X::char_type` is not the same type as `C`, the program is ill-formed.~~ `different_from<X::char_type, C>` is true, the program is ill-formed

The following is to be modified from 21.3.2.1 *General requirements* [**string.require**], statement 3

In every specialization `basic_string<charT, traits, Allocator>`, the type `allocator_traits<Allocator>::value_type` shall name the same type as `charT`. Every object of type `basic_string<charT, traits, Allocator>` uses an object of type `Allocator` to allocate and free storage for the contained `charT` objects as needed. The `Allocator` object used is obtained as described in [container.requirements.general]. In every specialization `basic_string<charT, traits, Allocator>`, the type `traits` shall mee the character traits requirements ([char.traits]). [Note: The program is ill-formed if ~~`traits::char_type` is not the same type as `charT`~~ `different_from<traits::char_type, charT> is true` —end note]

The following is to be modified from 21.4.2 Class template `basic_string_view` [string.view.template], statement 1

In every specialization `basic_string_view<charT, traits>`, the type `traits` shall mee the character traits requirements ([char.traits]). [Note: The program is ill-formed if ~~`traits::char_type` is not the same type as `charT`~~ `different_from<traits::char_type, charT> is true` —end note]

The following is to be removed from 24.5.1 Helper concepts [range.utility.helpers]

```
template<class T, class U>
    concept not_same_as = // exposition only
    !same_as<remove_cvref_t<T>, remove_cvref_t<U>>;
```

The following is to be modified in 24.5.3 Sub-ranges [range.subrange]

```
template<class From, class To>
    concept convertible-to-non-slicing = // exposition only
        convertible_to<From, To> &&
        !(is_pointer_v<decay_t<From>> &&
            is_pointer_v<decay_t<To>> &&
            not_same_as<remove_pointer_t<decay_t<From>>,
                remove_pointer_t<decay_t<To>>>>);
        distinct_from<remove_pointer_t<decay_t<From>>,
            remove_pointer_t<decay_t<To>>>);
```

The following is to be modified in 24.5.3.1 Constructors and conversions [range.subrange.ctor]

```

template<not same as<subrange> R>
template<distinct_from<subrange> R>
    requires borrowed_range<R> &&
           convertible-to-non-slicing<iterator_t<R>, I> &&
           convertible_to<sentinel_t<R>, S>
constexpr subrange(R&& r) requires (!StoreSize || sized_range<R>);

```

```

template<not same as<subrange> PairLike>
template<distinct_from<subrange> PairLike>
    requires pair-like-convertible-from<PairLike, const I&, const S&>
constexpr operator PairLike() const;

```

The following is to be modified in 24.7.3.1 *Class template ref_view* [**range.ref.view**]

Inside of statement 1

```

template<not same as<ref_view> T>
template<distinct_from<ref_view> T>
    requires /* see below */
constexpr ref_view(T&& t);

```

At the definition preceeding statement 2

```

template<not same as<ref_view> T>
template<distinct_from<ref_view> T>
    requires /* see below */
constexpr ref_view(T&& t);

```

The following is to be modified in 29.3.2 *Overview* [**iostream.forward.overview**], statement 6

[Note: For each of the class tempaltes above, the program is ill-formed if ~~traits::char_type is~~
~~not the same type as charT~~ different_from<traits::char_type, charT> is true
 ([char.traits]) —end note]

The following is to be modified in 32.4.2.2 *Constructors* [**thread.thread.constr**], statement 3

Constraints: ~~remove_cvref_t<F> is not the same type as thread~~ distinct_from<F,
thread> is true

The following is to be modified in 32.4.3.1 *Constructors, move, and assignment*, statement 3

Constraints: ~~remove_cvref_t<F> is not the same type as jthread~~ distinct_from<F, jthread> is true

The following is to be modified in 32.9.10.1 *Member functions* [**futures.task.members**], statement 2

Constraints: ~~remove_cvref_t<F> is not the same type as packaged_task<R(ArgTypes...)>~~ distinct_from<F, packaged_task> is true

The following is to be removed from the *Index of library concepts*

~~not-same-as[range.utility.helpers]~~

The following is to be added to the *Index of library concepts*

different_from[concept.different] , similar_to[concept.similar] ,
distinct_from[concept.distinct]